MANUAL TÉCNICO ESPECIALIZADO • VOLUME 2

Dominando o SQL: JOINs, Subqueries e Análises

Modelagem relacional, junções internas e externas comentadas linha a linha, subconsultas aninhadas e CTEs analíticas.

1. O Ecossistema Relacional: Normalização e Chaves

No Volume 1, operamos em uma única entidade isolada. Na vida real dos sistemas de alta escala, os dados são **normalizados** para evitar duplicidade, inconsistência e anomalias de atualização. As tabelas comunicam-se através de **Chaves Primárias (PK)** e **Chaves Estrangeiras (FK)**.

> **Esquema Oficial do Volume 2:** Expandimos nossa loja virtual criando as entidades `clientes`, `pedidos` e `itens_pedido`, associadas à tabela de `produtos`:

```sql
-- 1. Clientes cadastrados
CREATE TABLE clientes (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    cidade VARCHAR(60) NOT NULL,
    uf CHAR(2) NOT NULL
);

-- 2. Pedidos realizados
CREATE TABLE pedidos (
    id SERIAL PRIMARY KEY,
    cliente_id INT REFERENCES clientes(id) ON DELETE CASCADE,
    data_pedido DATE DEFAULT CURRENT_DATE,
    status VARCHAR(20) DEFAULT 'PENDENTE'
);

-- 3. Detalhes e itens por pedido (relação N:N entre Pedidos e Produtos)
CREATE TABLE itens_pedido (
    pedido_id INT REFERENCES pedidos(id) ON DELETE CASCADE,
    produto_id INT REFERENCES produtos(id),
    quantidade INT NOT NULL CHECK (quantidade > 0),
    preco_unitario DECIMAL(10, 2) NOT NULL,
    PRIMARY KEY (pedido_id, produto_id)
);
```

Tipo de Relação	Exemplo no Esquema	Regra de Integridade Referencial
1 : N (Um para Muitos)	`clientes` → `pedidos`	Um cliente pode possuir de zero a múltiplos pedidos vinculados.
N : N (Muitos para Muitos)	`pedidos` ↔ `produtos`	Resolvido através da tabela associativa intermediária `itens_pedido`.

2. A Família de JOINs: Conectando Tabelas

A instrução `JOIN` cruza linhas entre tabelas através de predicados definidos na cláusula `ON`. Abaixo, dissecamos as quatro modalidades fundamentais com códigos ricamente comentados.

2.1 INNER JOIN: A Interseção Exata

Retorna *apenas* os registros que possuem correspondência válida e confirmada em ambos os lados:

```sql
-- Recupera pedidos ativos exibindo nome do cliente e status
SELECT
    p.id AS pedido_numero,
    p.data_pedido,
    c.nome AS cliente_nome,
    c.cidade,
    p.status
FROM pedidos p
-- Liga a chave estrangeira do pedido à chave primária do cliente
INNER JOIN clientes c ON p.cliente_id = c.id
WHERE p.status <> 'CANCELADO';
```

2.2 LEFT (OUTER) JOIN: Preservando a Tabela da Esquerda

Mantém todas as linhas da tabela à esquerda (`FROM`), trazendo dados da tabela à direita quando houver, ou `NULL` quando inexistente:

```sql
-- Lista TODOS os clientes e identifica quem nunca comprou nada
SELECT
    c.id AS cliente_id,
    c.nome AS cliente_nome,
    COUNT(p.id) AS total_pedidos_realizados
FROM clientes c
-- LEFT JOIN garante que clientes sem pedidos não sejam descartados
LEFT JOIN pedidos p ON c.id = p.cliente_id
GROUP BY c.id, c.nome
-- Se desejar apenas clientes sem pedidos: WHERE p.id IS NULL
ORDER BY total_pedidos_realizados ASC;
```

2.3 RIGHT JOIN e FULL OUTER JOIN

```sql
-- FULL OUTER JOIN: Traz tudo de ambos os lados, preenchendo ausências com NULL
SELECT
    prod.nome AS produto_catalogo,
    it.pedido_id AS pedido_vinculado
FROM produtos prod
FULL OUTER JOIN itens_pedido it ON prod.id = it.produto_id;
```

> **Dica de Boas Práticas:** Em projetos profissionais, priorize o uso de `LEFT JOIN` no lugar de `RIGHT JOIN` por convenção de leitura ocidental (da esquerda para a direita).

3. Junções Múltiplas e Casos Avançados de JOIN

3.1 Junção Múltipla em Cadeia (Cruzando 4 Tabelas)

Em sistemas corporativos, relatórios analíticos exigem a reconstrução integral do carrinho de compras navegando por toda a árvore relacional:

```sql
-- Relatório Financeiro: Cliente + Pedido + Itens + Produto Original
SELECT
    c.nome AS cliente,
    p.id AS pedido_id,
    p.data_pedido,
    prod.nome AS produto_adquirido,
    it.quantidade,
    it.preco_unitario,
    (it.quantidade * it.preco_unitario) AS subtotal_item
FROM clientes c
-- 1º Salto: Identifica pedidos do cliente
INNER JOIN pedidos p ON c.id = p.cliente_id
-- 2º Salto: Abre a tabela associativa de itens do pedido
INNER JOIN itens_pedido it ON p.id = it.pedido_id
-- 3º Salto: Busca detalhes cadastrais do produto comprado
INNER JOIN produtos prod ON it.produto_id = prod.id
ORDER BY p.id DESC, subtotal_item DESC;
```

3.2 SELF JOIN: A Tabela Relacionada a Si Própria

Ocorre quando uma tabela possui chave estrangeira apontando para sua própria chave primária (ex.: colaboradores e seus gerentes, ou categorias-mãe e subcategorias):

```sql
-- Supondo que produtos possua a coluna 'produto_relacionado_id'
SELECT
    p.nome AS produto_principal,
    p.preco AS preco_principal,
    rel.nome AS item_recomendado_cross_sell,
    rel.preco AS preco_recomendado
FROM produtos p
INNER JOIN produtos rel ON p.produto_relacionado_id = rel.id;
```

3.3 CROSS JOIN: O Produto Cartesiano Matemático

```sql
-- Cruza todas as categorias com todos os trimestres fiscais para matriz de metas
SELECT p.categoria, t.trimestre
FROM (SELECT DISTINCT categoria FROM produtos) p
CROSS JOIN (VALUES ('Q1'), ('Q2'), ('Q3'), ('Q4')) AS t(trimestre);
```

> **Cuidado de Performance:** O `CROSS JOIN` entre duas tabelas de 10.000 linhas produz 100 milhões de registros na memória! Use-o apenas com filtros estritos ou tabelas de dimensões reduzidas.

4. Subqueries Aninhadas e Expressões de Tabela Comuns (CTEs)

Uma **subquery** é uma consulta embutida no interior de outra instrução SQL. Ela permite cálculos em dois estágios sem intervenção manual.

4.1 Subqueries Escalares (Filtro Dinâmico)

```sql
-- Seleciona produtos cujo preço supera a média de TODO o catálogo
SELECT nome, preco, categoria
FROM produtos
WHERE preco > (
    -- Subquery escalar: Retorna exatamente 1 valor numérico
    SELECT AVG(preco) FROM produtos
)
ORDER BY preco DESC;
```

4.2 Subqueries Correlacionadas com EXISTS / NOT EXISTS

Diferente da escalar, a subquery correlacionada é reavaliada para cada linha processada pela consulta principal, utilizando variáveis de fora:

```sql
-- Produtos que NUNCA foram comprados em nenhum pedido
SELECT p.id, p.nome, p.preco
FROM produtos p
WHERE NOT EXISTS (
    -- Valida se existe ao menos um registro correlacionado
    SELECT 1
    FROM itens_pedido ip
    WHERE ip.produto_id = p.id
);
```

4.3 CTEs (Common Table Expressions) com Cláusula WITH

As CTEs substituem subconsultas aninhadas complexas e ilegíveis por blocos declarativos limpos e reutilizáveis:

```sql
-- Cálculo de ticket médio por cliente e segmentação VIP
WITH faturamento_clientes AS (
    SELECT
        c.id AS cliente_id,
        c.nome,
        COALESCE(SUM(it.quantidade * it.preco_unitario), 0) AS total_gasto
    FROM clientes c
    LEFT JOIN pedidos p ON c.id = p.cliente_id
    LEFT JOIN itens_pedido it ON p.id = it.pedido_id
    GROUP BY c.id, c.nome
)
SELECT
    cliente_id, nome, total_gasto,
    CASE
        WHEN total_gasto >= 2000.00 THEN 'CLIENTE OURO'
        WHEN total_gasto > 0 THEN 'CLIENTE PADRÃO'
        ELSE 'PROSPECT SEM COMPRAS'
    END AS classificacao
FROM faturamento_clientes
ORDER BY total_gasto DESC;
```

5. Caderno Prático: Exercícios de Fixação (01 a 05)

Cenários analíticos do mundo corporativo envolvendo junções relacionais, filtros de integridade e subqueries.

Exercício 01 • Interseção Básica (INNER JOIN) BÁSICO

Enunciado: Escreva uma consulta que retorne o código do pedido (`id`), a data em que foi realizado e o nome do respectivo cliente, listando apenas os pedidos cujo status seja igual a `'FINALIZADO'`.

Exercício 02 • Detecção de Inatividade (LEFT JOIN com Filtro NULL) INTERMEDIÁRIO

Enunciado: O departamento de marketing precisa enviar um cupom de boas-vindas para usuários cadastrados que ainda não realizaram compras. Liste o `id`, `nome` e `cidade` de todos os clientes que possuem zero pedidos associados.

Exercício 03 • Carrinho Completo (JOIN Triplo) INTERMEDIÁRIO

Enunciado: Selecione o código do pedido, o nome do produto comercializado, a quantidade adquirida e o preço unitário praticado na venda. Ordene o resultado pelo código do pedido em ordem ascendente.

Exercício 04 • Valor Total por Pedido (JOIN + GROUP BY) INTERMEDIÁRIO

Enunciado: Gere um relatório com o código do pedido, a data da compra e o `valor_total_pedido` (calculado como a soma de quantidade vezes o preço unitário de cada item). Ordene do maior faturamento para o menor.

Exercício 05 • Filtro com Subquery Escalar INTERMEDIÁRIO

Enunciado: Localize o nome e o preço de todos os produtos cujo preço de venda seja estritamente superior ao preço médio de todos os produtos da categoria `'Periféricos'`.

5. Caderno Prático: Exercícios de Fixação (06 a 10)

Desafios avançados envolvendo subconsultas correlacionadas, operadores de conjunto, HAVING sobre junções e CTEs.

Exercício 06 • Subquery Correlacionada com NOT EXISTS AVANÇADO

Enunciado: Exiba o `id`, `nome` e `categoria` de todos os produtos do catálogo que nunca figuraram em nenhum item de pedido registrado na tabela `itens_pedido`, utilizando obrigatoriamente a cláusula `NOT EXISTS`.

Exercício 07 • Subquery com Operador IN em Conjuntos INTERMEDIÁRIO

Enunciado: Retorne os nomes de todos os clientes que já compraram ao menos um produto pertencente à categoria `'Monitores'`, utilizando uma subquery com o predicado `IN`.

Exercício 08 • Faturamento por Categoria (Junção Quádrupla + Agregação) AVANÇADO

Enunciado: Calcule a receita bruta total auferida por cada categoria de produto em pedidos concluídos (`status = 'FINALIZADO'`). Exiba o nome da categoria e o total faturado, ordenando pelo maior montante financeiro.

Exercício 09 • Clientes Recorrentes com HAVING em JOIN INTERMEDIÁRIO

Enunciado: Apresente o nome do cliente e a quantidade total de pedidos efetuados, exibindo apenas clientes que já realizaram 2 ou mais pedidos na plataforma comercial.

Exercício 10 • Desafio Corporativo com CTE (Common Table Expression) AVANÇADO

Enunciado: Utilizando a cláusula `WITH`, crie uma CTE chamada `compras_cliente` que calcule o gasto total acumulado de cada cliente. Na consulta principal, exiba os clientes que gastaram acima da média de gastos de todos os clientes compradores.

6. Gabarito Comentado: Exercícios 01 a 05

Solução 01 • INNER JOIN com Condição em Status

```sql
SELECT p.id AS pedido_id, p.data_pedido, c.nome AS cliente_nome
FROM pedidos p
INNER JOIN clientes c ON p.cliente_id = c.id
WHERE p.status = 'FINALIZADO';
```

Explicação: O `INNER JOIN` conecta a chave estrangeira `p.cliente_id` à chave primária `c.id`, excluindo qualquer pedido sem correspondência. O predicado `WHERE` filtra exclusivamente os concluídos.

Solução 02 • Identificação de Registros Órfãos via LEFT JOIN

```sql
SELECT c.id, c.nome, c.cidade
FROM clientes c
LEFT JOIN pedidos p ON c.id = p.cliente_id
WHERE p.id IS NULL;
```

Explicação: O `LEFT JOIN` preserva todos os clientes. Para aqueles que não possuem pedidos na tabela à direita, a coluna `p.id` é preenchida como `NULL`, permitindo isolar prospects sem histórico de compra.

Solução 03 • Associação em Cadeia (Pedidos, Itens e Catálogo)

```sql
SELECT ip.pedido_id, prod.nome AS produto, ip.quantidade, ip.preco_unitario
FROM itens_pedido ip
INNER JOIN produtos prod ON ip.produto_id = prod.id
ORDER BY ip.pedido_id ASC;
```

Explicação: Cruzamos a tabela associativa com os metadados do catálogo de produtos para obter a descrição legível do item juntamente aos dados históricos gravados no momento da venda.

Solução 04 • Valor Consolidado por Pedido

```sql
SELECT
    p.id AS pedido_id, p.data_pedido,
    SUM(ip.quantidade * ip.preco_unitario) AS valor_total_pedido
FROM pedidos p
INNER JOIN itens_pedido ip ON p.id = ip.pedido_id
GROUP BY p.id, p.data_pedido
ORDER BY valor_total_pedido DESC;
```

Explicação: O agrupamento reúne todos os itens pertencentes a um mesmo `pedido_id`, aplicando a função agregadora `SUM` sobre a multiplicação de volume e preço unitário.

Solução 05 • Comparação contra Média Dinâmica por Subquery

```sql
SELECT nome, preco, categoria
FROM produtos
WHERE preco > (
    SELECT AVG(preco)
    FROM produtos
    WHERE categoria = 'Periféricos'
);
```

Explicação: A subquery entre parênteses calcula em tempo de execução o preço médio de periféricos e retorna um único escalar, utilizado imediatamente pelo operador > da consulta externa.

6. Gabarito Comentado: Exercícios 06 a 10

Solução 06 • Subquery Correlacionada com NOT EXISTS

```sql
SELECT p.id, p.nome, p.categoria
FROM produtos p
WHERE NOT EXISTS (
    SELECT 1 FROM itens_pedido ip
    WHERE ip.produto_id = p.id
);
```

Explicação: Para cada produto da query principal, o motor verifica se há linhas em `itens_pedido` com o mesmo ID. O uso de `SELECT 1` otimiza a execução, pois o que importa é a existência do registro, não seus dados.

Solução 07 • Subquery com IN em Múltiplos Níveis

```sql
SELECT nome FROM clientes
WHERE id IN (
    SELECT p.cliente_id FROM pedidos p
    INNER JOIN itens_pedido ip ON p.id = ip.pedido_id
    INNER JOIN produtos prod ON ip.produto_id = prod.id
    WHERE prod.categoria = 'Monitores'
);
```

Explicação: A subquery gera a lista distinta de IDs de clientes que possuem produtos da categoria procurada em seus carrinhos, e o operador `IN` projeta apenas seus nomes.

Solução 08 • Faturamento por Categoria (4 Tabelas)

```sql
SELECT
    prod.categoria,
    SUM(ip.quantidade * ip.preco_unitario) AS receita_bruta
FROM produtos prod
INNER JOIN itens_pedido ip ON prod.id = ip.produto_id
INNER JOIN pedidos p ON ip.pedido_id = p.id
WHERE p.status = 'FINALIZADO'
GROUP BY prod.categoria
ORDER BY receita_bruta DESC;
```

Explicação: Navega por `produtos → itens_pedido → pedidos`, garantindo que compras pendentes ou canceladas sejam expurgadas da métrica financeira via cláusula `WHERE`.

Solução 09 • Filtragem Pós-Agregação com HAVING

```sql
SELECT c.nome, COUNT(p.id) AS total_pedidos
FROM clientes c
INNER JOIN pedidos p ON c.id = p.cliente_id
GROUP BY c.id, c.nome
HAVING COUNT(p.id) >= 2;
```

Explicação: `HAVING COUNT(p.id) >= 2` restringe a saída apenas aos clientes cujos grupos contêm dois ou mais pedidos associados.

Solução 10 • Arquitetura Analítica com CTE (WITH)

```sql
WITH faturamento_por_cliente AS (
    SELECT
        c.id, c.nome,
        SUM(ip.quantidade * ip.preco_unitario) AS gasto_total
    FROM clientes c
    INNER JOIN pedidos p ON c.id = p.cliente_id
    INNER JOIN itens_pedido ip ON p.id = ip.pedido_id
    GROUP BY c.id, c.nome
)
SELECT id, nome, gasto_total
FROM faturamento_por_cliente
WHERE gasto_total > (
    SELECT AVG(gasto_total) FROM faturamento_por_cliente
)
ORDER BY gasto_total DESC;
```

Explicação: A CTE encapsula a complexidade do cálculo de faturamento individual em uma tabela virtual. Em seguida, a query principal calcula a média global dos clientes compradores e filtra quem performou acima dessa média.